PHẢN HỒI BẢN RÀ SOÁT KỸ THUẬT VÒNG 2

Đội: Chiến binh Y tế số - ZeroCheck AI

Ngày rà soát của BTC: 31/07/2026

Ngày phản hồi: 31/07/2026

Đội cảm ơn Tổ giúp việc vì bản rà soát chi tiết và có căn cứ. Dưới đây là phản hồi theo đúng 10 mục ở phần B, kèm chỗ kiểm chứng cụ thể. **Cả 10 mục đội đều đã xử lý theo đề nghị của BTC**, kể cả hai mục ở bản dự thảo trước đội còn xin giải trình (viết lại lịch sử git, và đo tải trên cụm K8s). Ngoài ra đội **chủ động báo thêm ba sai sót của chính mình** mà bản rà soát chưa nêu: mật khẩu CSDL còn sót ở HEAD trong `docs/INSTALLATION.md`, một khóa API trong lịch sử git, và một file `.docx` của sản phẩm khác lẫn trong `docs/` làm phồng con số ảnh minh chứng.

Bảng tổng hợp

#	Nội dung	Ưu tiên BTC	Trạng thái
1	Mật khẩu CSDL trong `appsettings.json`	CAO	**Đã sửa ở HEAD + đã viết lại lịch sử git** + chặn tái diễn
2	`AllowAnyOrigin()` toàn API	CAO	Đã sửa
3	Hai repo không có test	CAO	Đã sửa - cả 3 repo có test và cổng chặn deploy
4	Có kịch bản k6 nhưng không có kết quả thô	TB	**Đã sửa - có kết xuất thô + đã đo trên cụm K8s thật**
5	Không có HPA / `.env.example`	TB	Đã sửa (HPA cho web-admin apply được ngay; HPA cho API kèm điều kiện)
6	Ảnh chôn trong `.docx`; loading/error	TB	Đã xuất ảnh; phần loading/error xin đính chính số liệu
7	web-admin không có prompt-log	TB	Đã sửa
8	Không có dependabot/workflow quét	THẤP	Đã sửa - đủ cả 3 repo
9	Một tác giả trên cả 3 repo	THẤP	Đội xác nhận: đội 3 người, chỉ 1 DEV (mục 9)
10	Không có cấu hình lint ở 2/3 repo	THẤP	Đã sửa

---

1. Mật khẩu CSDL trong `appsettings.json`

Đã làm:

- `appsettings.json` và `appsettings.Development.json` để chuỗi kết nối **rỗng**, kèm chú thích chỉ cách nạp bằng `dotnet user-secrets` hoặc biến môi trường

`ConnectionStrings__KnowledgeDatabase`.

- API **dừng ngay lúc khởi động** kèm thông báo tiếng Việt nếu thiếu chuỗi kết nối, thay vì chạy nửa vời với giá trị mặc định.
- Manifest `k8s/api-validation.yaml` đọc từ Secret `cbyts-secrets`; bổ sung `.env.example` liệt kê đủ biến; `.gitignore` của cả 3 repo đã chặn `.env`.
- Thêm workflow `security-scan.yml` (gitleaks, image ghim phiên bản) chạy mỗi push, mỗi pull request và một lượt định kỳ hằng tuần ở **cả 3 repo** - chặn bí mật mới lọt vào.

Đội xin chủ động báo thêm hai điều BTC chưa nêu:

1. **Mật khẩu CSDL còn sót ở HEAD, không chỉ trong lịch sử.** Khi rà lại theo đề nghị của BTC, đội phát hiện `docs/INSTALLATION.md` vẫn còn chuỗi kết nối kèm mật khẩu thật ngay tại bản hiện tại - tức là mục này nặng hơn mức BTC ghi nhận. Đã thay bằng chỗ điền `<mật khẩu PostgreSQL của bạn>` (commit riêng, 31/07/2026).
2. **Trong lịch sử còn một khóa API OpenRouter** ở `appsettings.Development.json` của một commit cũ (không còn ở HEAD, không còn trong thư mục làm việc). Đội **thu hồi khóa này trên openrouter.ai trước**, rồi mới gỡ khỏi lịch sử - đúng thứ tự BTC yêu cầu ở phần D.

Viết lại lịch sử git: đội chấp nhận tiêu chí của BTC ("xóa ở HEAD mà còn trong history thì chưa tính là khắc phục thật") và **thực hiện viết lại lịch sử** repo `hackathon-api-validation` bằng `git filter-repo --replace-text`, thay mọi lần xuất hiện của khóa API và mật khẩu CSDL trong toàn bộ lịch sử bằng chuỗi `*DA-GO-KHOI-LICH-SU*`. Trước khi chạy, đội đã tạo bản sao mirror của repo gốc để đối chiếu khi cần.

Hai hệ quả đội xin báo trước để BTC không hiểu nhầm là đội sửa hồ sơ:

- **Toàn bộ SHA commit của `hackathon-api-validation` thay đổi**, nên SHA `baa8479` ghi trong hồ sơ nộp bản trước không còn tra được. Nội dung cây mã **không đổi**, chỉ định danh đổi.
- **Tag `release-v2-final` đã được thay bằng `release-v2.1` ở cả 3 repo.** Riêng ở `hackathon-api-validation`, việc xóa tag cũ là **bắt buộc**: nó là tham chiếu duy nhất còn giữ nhánh lịch sử cũ sống trên GitHub - không xóa thì bí mật vẫn tra được bằng `git show release-v2-final:...` và việc viết lại lịch sử coi như vô hiệu. Đội làm theo hướng dẫn của BTC ("cứ tăng version lên"), nên mốc nộp lại của cả 3 repo là **`release-v2.1`**.
- Hai repo `hackathon-extension-his` và `hackathon-web-admin` đã được quét toàn bộ lịch sử theo 7 mẫu bí mật (khóa API, mật khẩu chuỗi kết nối, mật khẩu ứng dụng Gmail, kubeconfig, token) và **sạch**, nên **không** phải viết lại - SHA của hai repo này giữ nguyên.

Ghi chú kỹ thuật kèm theo, ở `api-validation/docs/PERFORMANCE-SECURITY.md`, mục "Quản lý bí mật và phụ thuộc".

2. CORS `AllowAnyOrigin()` áp toàn API

Đã sửa. Tách làm hai policy:

Policy	Áp cho	Origin
Mặc định	mọi endpoint quản trị: `/api/admin/*`, `/api/auth/*`, `/api/catalog`, `/api/dashboard`, `/api/icd`, `/api/legal`	đúng danh sách trong cấu hình `Cors:WebAdminOrigins`, đọc từ biến môi trường khi triển khai
`"Extension"`	gắn `[EnableCors("Extension")]` **đích danh 4 controller** extension gọi tới: `ExtensionConfigController`, `AssistantController`, `ValidationController`, `Xml130Controller`	mở origin

Bốn controller kia vẫn phải mở vì content script chạy với origin `chrome-extension://<id>`, mà `<id>` đổi theo từng máy cài khi load unpacked - không thể khai một danh sách cứng.

Xác nhận cơ chế lưu token của web-admin (BTC hỏi): web-admin lưu access/refresh token trong `localStorage` (`src/app/core/token.store.ts`) và tự đính vào header `Authorization`. **Không dùng cookie phiên**, nên trình duyệt không tự gửi kèm chứng thực - không có bề mặt CSRF qua cookie. Đây cũng là lý do `AllowCredentials` cố ý **không** bật. Có unit test `token.store.spec.ts` khẳng định token nằm ở `localStorage` và không xuất hiện trong `document.cookie`, để sau này ai đổi sang cookie thì test đỏ ngay.

Trên production web-admin gọi API **cùng origin** qua Ingress, nên CORS chỉ thực sự có tác dụng ở môi trường dev và với extension.

Giải thích đầy đủ ở `docs/PERFORMANCE-SECURITY.md`, mục "CORS: vì sao tách hai policy".

3. Kiểm thử ở cả 3 repo

Đã sửa. Chạy thật ngày 31/07/2026:

Repo	Khung test	Số test	Cổng chặn deploy
hackathon-api-validation	xUnit	**27**	job `test` chạy `dotnet test`, `build-and-deploy` khai `needs: test`
hackathon-extension-his	vitest	**38** (4 file: `rules`, `RuleEngine`, `ValidationRequestBuilder`, `BatchCapacity`)	job CI chạy `tsc --noEmit` + `eslint` + `vitest run`
hackathon-web-admin	vitest + jsdom	**16** (2 file: `token.store`, `auth.guard`)	job `test`, `build-and-deploy` khai `needs: test`

Tổng **81 test**, đều chạy trong CI, test đỏ là không deploy. Phần extension phủ đúng chỗ BTC chỉ ra: `content/validation/rules/` và `ValidationRequestBuilder`.

4. Kết quả k6 thô

Đã sửa phần kết quả thô. Repo `hackathon-api-validation`, thư mục `loadtest/`, có 4 file do chính k6 sinh bằng `--summary-export` (không phải bảng gõ tay):

File	Bản build	Kết quả
`ket-qua-release-1vu.json`	Release	p95 **172,94 ms**, p99 200,38 ms, trung vị 124,78 ms, 341 lượt, 0% lỗi
`ket-qua-release-20vu.json`	Release	p95 **1,85 s**, p99 2,12 s, **15,57 hồ sơ/giây**, 947 lượt, 0% lỗi
`ket-qua-1vu.json`	Debug (đối chứng)	p95 167,65 ms
`ket-qua-20vu.json`	Debug (đối chứng)	p95 1,77 s, 16,43 hồ sơ/giây

Hai lần chạy lệch nhau ~3% - chênh lệch nằm trong nhiễu đo, không phải do cấu hình build.

Đính chính số cũ, do đội chủ động: tài liệu và slide bản trước ghi p95 147 ms / 20 VU 1,45 s. Đó cũng là số đo thật nhưng của một lần chạy **không lưu file kết xuất**, nên không kiểm chứng lại được. Toàn bộ tài liệu, slide, trang giới thiệu và thẻ kết video đã được chỉnh về con số **thận trọng hơn** có file gốc đi kèm.

Điều kiện đo ghi rõ trong `docs/PERFORMANCE-SECURITY.md`: một máy để bàn Windows 11 chạy đồng thời cả API, k6 và PostgreSQL; Kestrel `http://localhost:5075`; 3 hồ sơ XML 130 **thật** cỡ 7 / 57 / 108 KB; đường đo `POST /api/xml130/validate?useAi=false`. Đây là **cận dưới** của năng lực thật.

Đã làm - đo trên cụm K8s thật (31/07/2026): theo đề nghị của BTC, đội đã đo trên **chính cụm VNPT Cloud**. Cách làm: dựng namespace riêng `cbyts-loadtest` - cùng image, cùng `resources.limits`, có PostgreSQL riêng - chạy k6 **bên trong cụm**, đo xong xóa cả namespace. Manifest dựng lại toàn bộ môi trường đo nằm ở `k8s/loadtest/` kèm README hướng dẫn chạy lại.

Lượt đo	Replica API	p95	p99	Trung vị	Thông lượng	Lỗi HTTP	Kết xuất thô
1 VU / 45 s	1	**251 ms**	345 ms	194 ms	5,00 hồ sơ/s	**0%**	`loadtest/ket-qua-cum-1vu.json`
20 VU / 60 s	1	6,99 s	9,38 s	5,49 s	3,63 hồ sơ/s	**0%**	`loadtest/ket-qua-cum-20vu.json`
20 VU / 60 s	**3**	3,87 s	5,19 s	**1,44 s**	**10,83 hồ sơ/s**	**0%**	`loadtest/ket-qua-cum-20vu-3replica.json`

Cả ba lượt đều 0% lỗi HTTP và 100% check nội dung phản hồi đạt (450/450, 456/456, 1320/1320).

Hai điều đội xin nói rõ để BTC đọc số cho đúng:

1. **Một pod trên cụm chậm hơn máy dev - đúng như khai báo, không phải cụm yếu.** Manifest ghi `limits.cpu: "1"`, mỗi pod chỉ được một CPU; máy dev nhiều nhân nên phục vụ song song tốt hơn. So sánh đúng phải là *một pod 1 CPU* với *một máy nhiều nhân*. Đội **đính chính** một câu trong tài

liệu bản trước có ý "trên node K8s riêng sẽ tốt hơn" - phép đo này cho thấy điều ngược lại, và đội đã sửa lại tài liệu theo số thật.

2. **Scale ngang tuyến tính - nay là số đo, không còn là suy luận kiến trúc.** Giữ nguyên tải 20 VU, chỉ tăng từ 1 lên 3 replica: thông lượng 3,63 → 10,83 hồ sơ/giây (**gấp 2,98 lần**), trung vị 5,49 s → 1,44 s (**nhanh 3,8 lần**). Đây chính là căn cứ thực nghiệm cho HPA ở mục 5.

Hai ranh giới đội vẫn giữ, xin BTC ghi nhận:

- Đội **không** bắn tải vào namespace `cbyts` đang phục vụ thật. Namespace đó **không nhận một request nào** của phép đo. Quy tắc này ghi ngay trong header file `loadtest/xml130-validate.k6.js`. Chạy k6 *bên trong* cụm còn cho số sạch hơn: không lẫn độ trễ WAN và hàng đợi load balancer, nên đo đúng năng lực xử lý.
- Dữ liệu đưa lên cụm là bản **đã ẩn danh**: họ tên, địa chỉ, điện thoại, số thẻ, mã bệnh nhân thay bằng ký tự giả **giữ nguyên độ dài**, nên khối lượng phân tích không đổi mà hồ sơ bệnh nhân thật không rời máy dev.

Ngưỡng chống spam được nâng tạm **chỉ trong namespace đo tải** (initContainer ghi `Storage/ratelimit-settings.json`); để mặc định 100 req/phút/IP thì phép đo chỉ đo được bộ chống spam. Production giữ nguyên 100 req/phút/IP.

5. HPA, resources, `.env.example`

Đã sửa:

- `api-validation/k8s/hpa.yaml`: HPA cho **web-admin** (2-6 replica, CPU 70%, `stabilizationWindowSeconds: 300`) - apply được ngay vì web-admin là nginx phục vụ file tĩnh, stateless hoàn toàn.
- HPA cho **api-validation** để sẵn trong cùng file nhưng **đang comment**, kèm ghi rõ hai điều kiện phải làm trước: (a) đổi PVC `api-storage` từ ReadWriteOnce sang ReadWriteMany, (b) tách migration EF Core ra initContainer/Job. Đội chọn ghi thẳng điều kiện thay vì apply một manifest sẽ hỏng khi scale.
- `.env.example` đã có ở `api-validation`.

Xin đính chính một điểm nhỏ: `resources.requests/limits` **đã** được khai từ trước cho các container api-validation, web-admin, postgres và ollama - xem `k8s/api-validation.yaml` (api: requests 200m/256Mi, limits 1 CPU/1Gi), `k8s/postgres.yaml`, `k8s/ollama.yaml`, `web-admin/k8s/web-admin.yaml`. Có thể mục này bị gộp chung với phần HPA khi rà.

6. Ảnh trong `.docx`, video, loading/error

Đã làm - xuất ảnh: `api-validation/docs/screenshots/` gồm **42 ảnh PNG rời**, đặt tên theo dạng `<số thứ tự>-<mục trong tài liệu>.png` nên đọc lần lượt là đi đúng luồng thao tác:

Thư mục	Số ảnh	Nội dung
`screenshots/web-admin/`	19	đăng nhập, dashboard, kho tri thức, văn bản pháp quy, danh mục HIS, người dùng, vai trò & quyền, nhật ký kiểm toán, cấu hình AI / robot / hệ thống

Thư mục	Số ảnh	Nội dung
`screenshots/extension-chrome/`	23	cài đặt extension và thao tác thật trên màn hình HIS

Đội xin chủ động báo một sai sót của chính mình: trong con số "344 ảnh" mà BTC ghi nhận là điểm mạnh, có **302 ảnh đến từ** `docs/hdsd-webadmin.docx` - file này **không phải** tài liệu của ZeroCheck AI. Nó có trang bìa của sản phẩm nhưng toàn bộ phần ruột là hướng dẫn của một sản phẩm VNPT khác ("Hệ thống điều hành khu phố ấp": quản lý hộ dân, cư dân, cuộc họp, phổ cập giáo dục). Đây là file mẫu bị lẫn vào thư mục `docs/`, không phải cố ý. Đội đã **xóa hẳn file này khỏi repo** và **không** xuất 302 ảnh đó ra; đề nghị BTC **bỏ file này khỏi phần đánh giá TC4**. Số liệu đúng của đội là **42 ảnh thật** của sản phẩm.

Xin đính chính về loading/error state: nhận định "0 file có trạng thái loading/error" trong mã extension không khớp với mã nguồn. Toàn bộ lời gọi AI của extension đi qua hai lớp và cả hai đều có xử lý chờ lẫn báo lỗi bằng tiếng Việt:

Chỗ	Bằng chứng
`content/chat/ChatPanel.ts`	`setBusy()` khóa toàn bộ nút hành động, chỉ chừa nút Dừng ■, đổi placeholder thành "Đang xử lý - bấm ■ để dừng..."; `addLoadingMessage()` chèn bong bóng chờ; kiểm tra lô hiển thị tiến độ chạy chữ theo từng hồ sơ ("Đang kiểm tra hồ sơ 12/57...", "Đang đối chiếu chéo giữa các hồ sơ..."); hỏi đáp AI hiện "Đang tìm câu trả lời..."; **6 khối** `try/catch` bọc quanh các lời gọi API
`content/chat/ChatApiClient.ts`	phân biệt **3 kiểu kết thúc** và trả thông điệp tiếng Việt riêng cho từng kiểu: hết giờ ("Máy chủ xử lý quá lâu (quá N giây)..."), người dùng chủ động dừng (không hiển thị như lỗi), lỗi kết nối ("Không kết nối được máy chủ..."); riêng HTTP 413 có hướng dẫn chia nhỏ lô theo tuần

Lý do phép rà tự động không thấy: mã nguồn đặt tên định danh và thông điệp bằng **tiếng Việt** theo quy ước bắt buộc của dự án (`CLAUDE.md`), nên không khớp từ khoá `loading` / `error`.

Về `@media`: đúng là 0 - và như chính BTC đã lưu ý, extension là UI chèn vào trang HIS sẵn có nên không đặt breakpoint là hợp lý về bản chất.

Video demo: đang quay theo kịch bản đã dựng (`nop-btc/09-video-demo/`), nộp cùng hồ sơ.

7. Prompt-log cho web-admin

Đã sửa. Thêm `web-admin/docs/AI-EVIDENCE.md` gồm 4 phần: (1) vị trí của web-admin trong hệ AI kèm bảng màn hình - endpoint và ranh giới quyền của AI, (2) AI được dùng thế nào khi phát triển, (3) **nhật ký prompt trích nguyên văn** từ phiên làm việc, giữ nguyên lỗi chính tả của người dùng, có ngày tháng trong khoảng 17/07 - 29/07 khớp lịch sử commit, (4) những gì con người kiểm lại chứ không giao cho AI.

`web-admin/README.md` cũng đã viết lại từ bản mẫu Angular CLI thành README dự án thật.

8. Cấu hình bảo mật trên GitHub

Đã sửa, đủ cả 3 repo:

- `.github/dependabot.yml`: quét nuget/npm/docker hằng tuần, github-actions hằng tháng; gom `@angular/*`, `Microsoft.*` / `System.*`, vite/vitest vào từng PR chung để đỡ nhiễu.
- `.github/workflows/security-scan.yml`: gitleaks (image ghim `v8.28.0`) chạy mỗi push, mỗi pull request và định kỳ hằng tuần.
- **Chặn thư viện dính CVE ngay trong CI**: `dotnet list package --vulnerable --include-transitive` (api), `npm audit --omit=dev --audit-level=high` (web-admin, extension).

Trong lúc dựng các cổng này, CI đã bắt được một lỗ hổng thật: `Microsoft.OpenApi` 2.0.0 kéo theo qua `Microsoft.AspNetCore.OpenApi`, dính GHSA-v5pm-xwqc-g5wc mức cao - đã ghim thẳng lên 2.7.6. Phụ thuộc phát hành của web-admin và extension hiện **0 lỗ hổng**.

Về quyền truy cập: đội đã mời tài khoản BTC vào cả 3 repo private. Nếu Tổ giúp việc chưa nhận được lời mời, xin báo lại để đội gửi lại.

9. Một tác giả trên cả 3 repo

Đội xác nhận: nhận định của BTC là đúng, và đây là hệ quả trực tiếp của cơ cấu đội.

Team có 3 thành viên, nên chỉ có 1 DEV, vì thế chỉ có 1 người commit. Các vị trí còn lại gồm 1 PM, 1 BA; đồng thời 2 vị trí này đều nhận nhiệm vụ tester.

Nói cách khác, một tác giả trên cả 3 repo **không** phản ánh việc mã nguồn do một người sinh ra mà không ai kiểm - nó phản ánh việc đội chỉ có một người giữ vai trò lập trình. Phần kiểm thử, nghiệm thu kịch bản nghiệp vụ và đối chiếu với quy định là do PM và BA thực hiện; những nội dung này không để lại dấu vết commit vì không phải là mã nguồn.

Về việc **web-admin được dựng mới bằng Angular CLI 22.0.7**: dấu vết `ng new` còn nguyên trong repo - cấu trúc thư mục Angular chuẩn, `angular.json` mặc định, và dòng ghi rõ phiên bản CLI ở cuối `README.md`. Đây là lý do lịch sử commit của repo này bắt đầu bằng một commit lớn duy nhất.

10. Cấu hình lint

Đã sửa:

Repo	Đã thêm	CI
hackathon-api-validation	`.editorconfig` ở gốc repo (quy ước C# + naming rule + `dotnet_diagnostic.*` severity); bật `EnableNETAnalyzers`, `AnalysisLevel=latest`, `EnforceCodeStyleInBuild` trong `.csproj`	bước "Kiểm tra quy ước code" chạy `dotnet format style` và `dotnet format analyzers` với `--verify-no-changes --severity error`
hackathon-extension-his	`eslint.config.js` (flat config, `typescript-eslint`); script `npm run lint`	bước "Kiểm tra ESLint" chạy thật, **không** dùng cờ bỏ qua lỗi
hackathon-web-admin	đã có `.editorconfig` + `.prettierrc` từ trước	-

Không dùng cờ nới lỏng. Việc dựng cổng này đã bắt được và sửa 3 lỗi thật trong mã extension (biểu thức ba ngôi viết như câu lệnh ở `ChatPanel.ts`, `UnifiedAssistantPanel.ts`, `PaymentListSniffer.ts`), và một cảnh báo bảo mật CA2100 trong `PgVectorEmbeddingIndex.cs` - chỗ này sau khi rà thì đúng là an toàn (DDL khởi tạo pgvector, chỉ ghép số chiều kiểu `int` đọc từ cấu hình, không có đầu vào người dùng) nên đội **ghi rõ lý do bằng `[SuppressMessage]` có `Justification`** thay vì tắt rule.

Một điểm đội xin BTC lưu ý khi chấm

Mốc nộp lại của cả 3 repo là tag `release-v2.1`, không còn `release-v2-final`.

Nguyên nhân: để gỡ bí mật khỏi lịch sử (mục 1), repo `hackathon-api-validation` phải viết lại lịch sử, nên **toàn bộ SHA commit của repo này đã đổi** và tag cũ - vốn là tham chiếu duy nhất còn giữ nhánh lịch sử cũ sống trên GitHub - buộc phải xóa. Đội làm theo hướng dẫn "cứ tăng version lên" của BTC và đặt `release-v2.1` cho cả 3 repo để đồng bộ. Nội dung cây mã không đổi, chỉ định danh đổi. Nếu BTC cần đối chiếu với bản trước khi viết lại, đội còn giữ bản sao mirror và sẵn sàng cung cấp.

Đội xin cảm ơn Tổ giúp việc. Bản rà soát đã chỉ ra đúng chỗ yếu nhất của hồ sơ là **sự không đồng đều giữa 3 repo** - đó cũng là phần đội tập trung sửa trong đợt này. Hai đề nghị ban đầu đội định xin giải trình (viết lại lịch sử git, đo tải trên cụm) sau khi cân nhắc lại đội đã **làm theo BTC**, và cả hai đều cho kết quả tốt hơn dự tính: lịch sử git nay sạch bí mật, còn phép đo trên cụm cho ra bằng chứng mạnh nhất của cả hồ sơ - **scale ngang 2,98 lần đo được bằng số**.